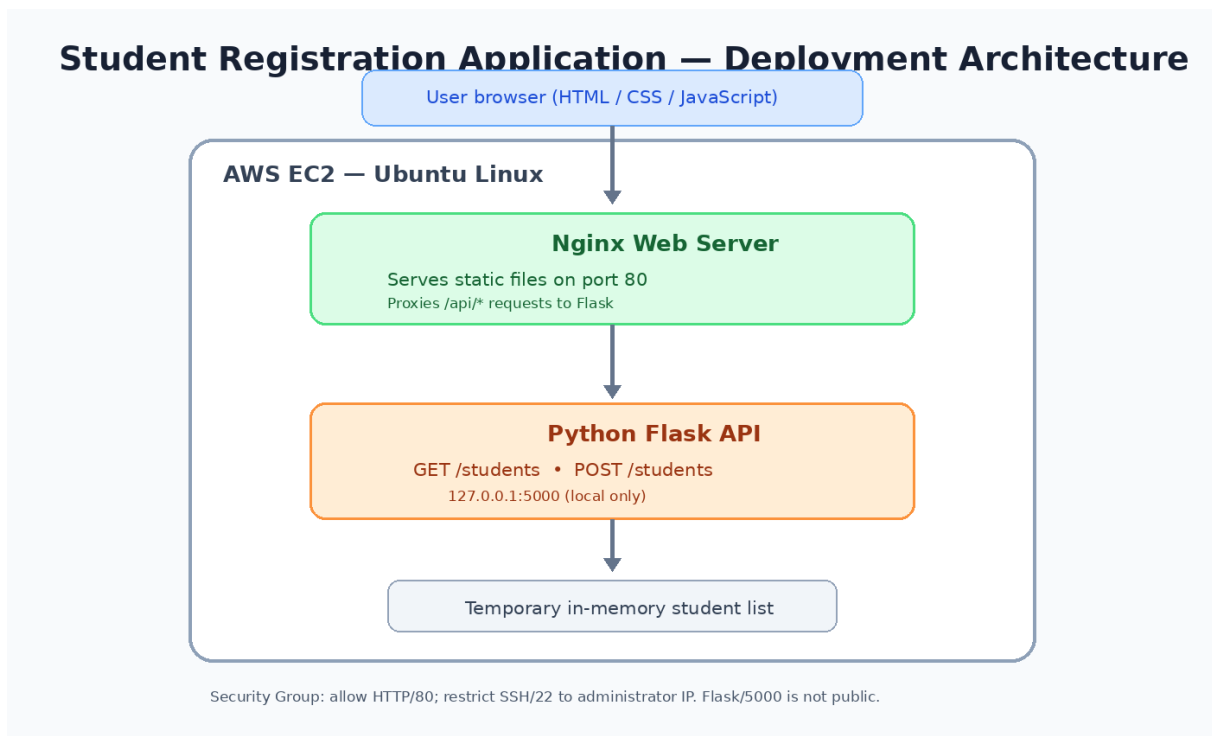


PROJECT REPORT

STUDENT REGISTRATION WEB APPLICATION

Frontend hosted with Nginx and backend powered by Flask on AWS EC2



Prepared as a beginner cloud deployment project

Name: _____

Date: _____

Contents

1. 1. Abstract
2. 2. Introduction
3. 3. Project Objectives
4. 4. Scope and Requirements
5. 5. Technologies Used
6. 6. System Architecture
7. 7. Application Modules
8. 8. Implementation
9. 9. API Design
10. 10. Testing and Results
11. 11. Security Considerations
12. 12. Limitations and Future Enhancements
13. 13. Conclusion
14. 14. References

Note: The screenshots/visuals included in this report are illustrative mockups prepared for documentation. Replace them with screenshots from your actual EC2 deployment if your instructor or interviewer requires evidence of a live run.

1. Abstract

This project demonstrates a small web application deployed on an Ubuntu server in Amazon Web Services (AWS) EC2. The frontend is built using standard HTML, CSS, and JavaScript and is served by Nginx. The backend is a Python Flask application that exposes REST endpoints for listing and registering students. Nginx serves the static website over HTTP and reverse-proxies requests under the `/api/` path to the Flask process running locally on port 5000. The design introduces key cloud and web-hosting concepts including Linux administration, web-server configuration, API communication, port management, and basic network security.

2. Introduction

A web application commonly separates its user interface from its server-side logic. The browser displays the interface, while an API processes requests and returns data. In this project, Nginx is responsible for delivering frontend files and forwarding API requests to a Flask backend. Both services can run on the same EC2 virtual machine, keeping the setup simple and suitable for learning.

The application allows a user to enter a student's name and email address, submit the form, and view the list of students currently held by the backend. The initial version stores records in memory only and is intended for demonstration.

3. Project Objectives

- Create a simple responsive frontend using HTML, CSS, and JavaScript without a frontend framework.
- Host the static website using Nginx on Ubuntu EC2.
- Build REST API endpoints with Python Flask.
- Configure Nginx reverse proxying so the browser uses one public web origin.
- Understand EC2 Security Groups, Linux services, logs, and basic troubleshooting.
- Validate communication between the frontend and backend.

4. Scope and Requirements

4.1 Functional scope

- Display a student registration form.
- Accept a student's name and email address.
- Send student data to the backend through an HTTP POST request.
- Retrieve and display the current student list through an HTTP GET request.
- Display a success or failure message to the user.

4.2 Software and infrastructure requirements

Component	Purpose
AWS EC2 Ubuntu	Virtual machine hosting the web and API services
Nginx	Static file server and reverse proxy
Python 3 + Flask	Backend REST API
HTML / CSS / JavaScript	Browser interface and API requests
SSH client and key pair	Secure server administration
Web browser + curl	Functional and endpoint testing

5. Technologies Used

- **Amazon EC2:** Provides an on-demand virtual server in the cloud.
- **Ubuntu Linux:** Operating system used to configure and run the services.
- **Nginx:** Serves index.html and style.css, and proxies /api/ requests.
- **HTML:** Defines the structure of the registration page.
- **CSS:** Styles the form, buttons, and student list.
- **JavaScript:** Sends fetch() requests to the API and updates the page.
- **Python Flask:** Implements the student API endpoints.

6. System Architecture

The user's browser requests the website from the public EC2 address over HTTP port 80. Nginx serves static files from /var/www/student-app. Requests starting with /api/ are proxied to Flask at 127.0.0.1:5000. Flask processes the request and returns JSON, which JavaScript uses to refresh the student list.

Student Registration Application — Deployment Architecture

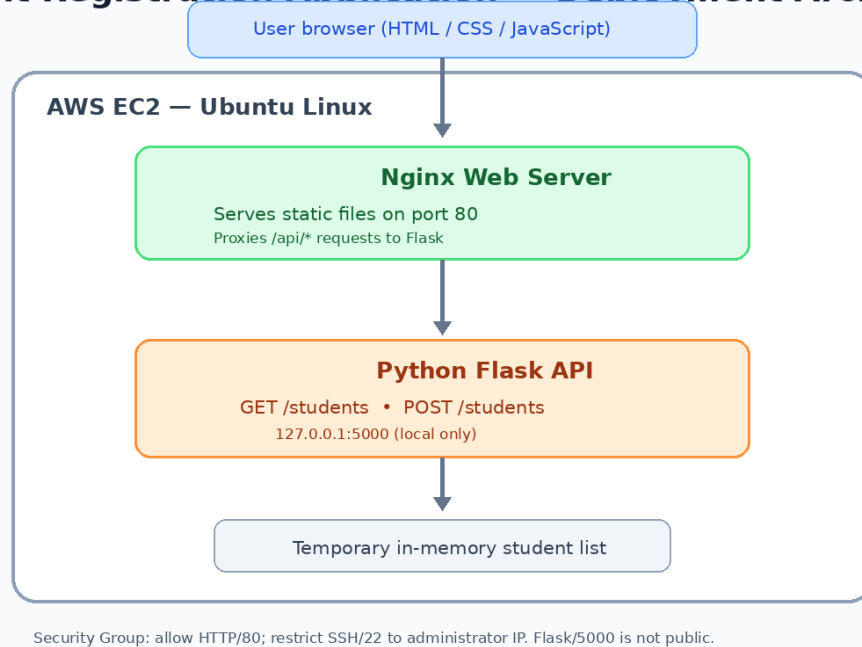
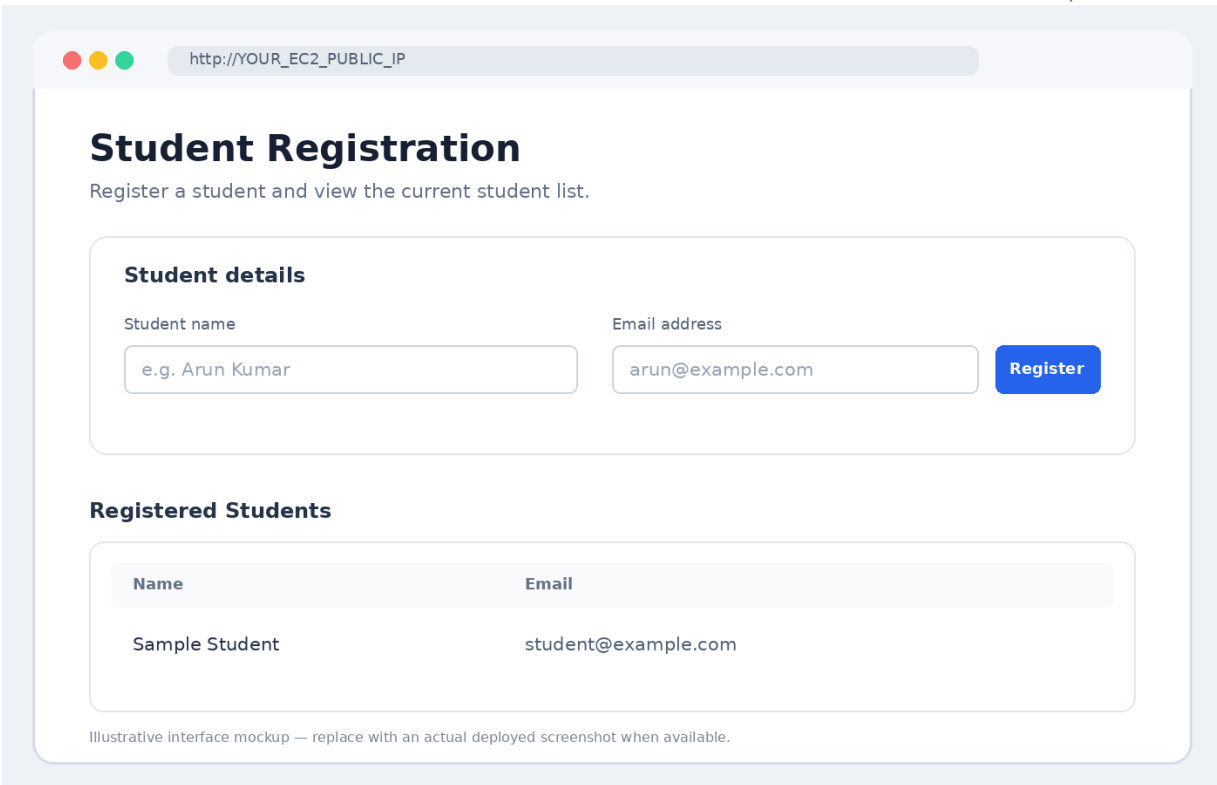


Figure 1. Project architecture (illustrative diagram).

7. Application Modules

7.1 Frontend module

The frontend consists of index.html and style.css. JavaScript listens for form submission, sends JSON to the API using fetch(), and updates the page without a full browser reload. The frontend calls /api/students rather than hard-coding the backend port or public IP address.



The mockup shows a web browser window with the address bar displaying 'http://YOUR_EC2_PUBLIC_IP'. The page title is 'Student Registration'. Below the title is a subtitle: 'Register a student and view the current student list.' The main content area is divided into two sections. The first section, 'Student details', contains two input fields: 'Student name' with the placeholder text 'e.g. Arun Kumar' and 'Email address' with the placeholder text 'arun@example.com'. A blue 'Register' button is positioned to the right of the email field. The second section, 'Registered Students', contains a table with two columns: 'Name' and 'Email'. The table has one row with the values 'Sample Student' and 'student@example.com'. At the bottom of the mockup, there is a note: 'Illustrative interface mockup — replace with an actual deployed screenshot when available.'

Figure 2. Illustrative student registration interface; not a live EC2 screenshot.

7.2 Nginx module

Nginx listens on port 80 and uses `/var/www/student-app` as its document root. The root location serves static files. The `/api/` location forwards requests to the local Flask process. This arrangement avoids exposing Flask's port directly to the public internet.

```
ubuntu@ec2: /etc/nginx/sites-available/student-app

server {
    listen 80;
    server_name _;
    root /var/www/student-app;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:5000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Figure 3. Nginx configuration illustration.

7.3 Backend module

The Flask backend provides two endpoints. GET /students returns the current list as JSON. POST /students validates that name and email are present, creates a simple record, and returns it with HTTP status 201. For this learning version, the student list is a Python in-memory list.

8. Implementation

15. **Provision EC2:** Launch an Ubuntu EC2 instance, create/select a key pair, and connect over SSH.
16. **Install packages:** Run apt update and install Nginx, Python 3, pip, and venv.
17. **Create frontend files:** Place index.html and style.css under /var/www/student-app.
18. **Configure Nginx:** Create a site file in /etc/nginx/sites-available/, enable it, validate with nginx -t, and reload Nginx.
19. **Create Flask API:** Create a Python virtual environment, install Flask, and add GET/POST endpoints.
20. **Run and test backend:** Run Flask on 127.0.0.1:5000 and verify the API locally with curl.
21. **Test through Nginx:** Open the EC2 public IP in a browser and register a sample student.

8.1 Example Nginx configuration

```
server {
    listen 80;
    server_name _;

    root /var/www/student-app;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:5000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

The trailing slash in proxy_pass means that the /api/ prefix is replaced when Nginx forwards the request. Therefore, a browser request to /api/students reaches Flask as /students.

9. API Design

Method	Path through Nginx	Purpose	Expected response
GET	/api/students	Retrieve student records	200 OK with JSON array
POST	/api/students	Register a student	201 Created with student JSON

Example POST body:

```
{"name": "Arun Kumar", "email": "arun@example.com"}
```

10. Testing and Results

The following test plan can be used to verify the deployment. Record the actual result after running each test on the EC2 instance.

Test	Command / action	Expected result
Nginx service	<code>sudo systemctl status nginx</code>	Nginx is active
Nginx config	<code>sudo nginx -t</code>	Syntax is ok; test is successful
Frontend	Open <code>http://EC2_PUBLIC_IP</code>	Registration page loads
API through proxy	<code>curl http://127.0.0.1/api/students</code>	JSON array, initially []
Register student	Submit the form in browser	Success message and student shown
Backend binding	<code>ss -lntp grep 5000</code>	Flask listens on localhost:5000

Expected results are documented here; this report does not claim that these checks were run in the user's live AWS account. Attach actual screenshots of the browser, terminal, and AWS configuration after verifying the deployment.

11. Security Considerations

- Allow inbound HTTP port 80 in the EC2 Security Group for public website access.
- Restrict SSH port 22 to the administrator's IP address rather than opening it to everyone.
- Bind Flask to 127.0.0.1 so clients cannot access port 5000 directly from outside the EC2 instance.
- Use HTTPS with a valid TLS certificate before collecting real user information.
- Do not store passwords or sensitive student data in this demonstration application.
- For a public deployment, add validation, logging, rate limiting, and a persistent database.

12. Limitations and Future Enhancements

12.1 Current limitations

- Student records are stored in memory and disappear when Flask restarts.
- The demonstration does not include authentication or role-based access control.
- The example is intended for learning, not production use.
- A production deployment should manage Flask with systemd or a production WSGI server rather than a development server.

12.2 Future enhancements

- Add SQLite or Amazon RDS to persist student records.
- Configure HTTPS using a domain name and TLS certificate.
- Run Flask with Gunicorn and manage it using systemd.
- Add update and delete operations, form validation, and structured logs.
- Add monitoring and backups; optionally package the services into Docker containers.

13. Conclusion

The Student Registration Web Application is a beginner-friendly example of connecting a static frontend to a backend API on AWS EC2. HTML, CSS, and JavaScript provide the user interface, Nginx serves the website and reverse-proxies API calls, and Flask processes the student requests. The project provides practical experience with Linux, cloud networking, web-server configuration, HTTP, REST APIs, and deployment troubleshooting. A database, HTTPS, and production process management are recommended next steps.

14. References

- Amazon EC2 documentation: <https://docs.aws.amazon.com/ec2/>

- Nginx documentation: <https://nginx.org/en/docs/>
- Flask documentation: <https://flask.palletsprojects.com/>
- Python documentation: <https://docs.python.org/3/>